

Implementación de Autenticación con JSON Web Tokens (JWT)

Resumen

La autenticación con JSON Web Token (JWT) es una técnica fundamental para proteger aplicaciones web modernas. Este método permite verificar la identidad de los usuarios y controlar el acceso a recursos específicos de manera eficiente y segura. A continuación, exploraremos cómo implementar este sistema de autenticación en una aplicación Node.js, desde la instalación de las dependencias necesarias hasta la creación de rutas protegidas.

¿Cómo implementar autenticación con JSON Web Token?

Para implementar un sistema de autenticación basado en JWT, necesitamos instalar algunas dependencias que nos facilitarán el trabajo. Estas herramientas nos permitirán crear usuarios, autenticarlos y generar tokens que controlarán el acceso a rutas protegidas.

Lo primero es instalar las bibliotecas necesarias:

A screenshot of a terminal window with a dark background. It shows two lines of command text: 'npm install jsonwebtoken' and 'npm install bcryptjs'. A small cursor icon is visible at the end of the second line.

```
npm install jsonwebtoken
npm install bcryptjs
```

La primera biblioteca nos permitirá trabajar con los tokens JWT, mientras que la segunda nos ayudará a encriptar las contraseñas de los usuarios para almacenarlas de forma segura en nuestra base de datos.

Creación del middleware de autenticación

El siguiente paso es crear un middleware que verificará la validez de los tokens en cada solicitud a rutas protegidas. Para ello, crearemos un archivo llamado auth.js en la carpeta de middlewares:

```
const JWT = require('jsonwebtoken');

function authenticateToken(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: "Access denied. No token provided." });
  }

  JWT.verify(token, process.env.JWT_SECRET, (error, user) => {
    if (error) return res.status(403).json({ error: "Invalid token" });

    req.user = user;
    next();
  });
}

module.exports = authenticateToken;
```

Este middleware realiza las siguientes acciones:

1. **Extrae el token** del encabezado de autorización
2. **Verifica si existe un token** en la solicitud
3. **Valida el token** utilizando la clave secreta almacenada en las variables de entorno
4. Si el token es válido, **almacena la información del usuario** en el objeto de solicitud y permite que la petición continúe

Es importante configurar la variable de entorno JWT_SECRET en nuestro archivo de configuración:

configuración:

```
JWT_SECRET=Platzi
```

Esta clave secreta debe ser segura y única para cada aplicación, ya que es fundamental para la seguridad del sistema de autenticación.

Implementación de rutas protegidas

Una vez que tenemos nuestro middleware de autenticación, podemos implementarlo en las rutas que queremos proteger. Para ello, primero importamos el middleware en nuestro archivo principal:

```
const authenticateToken = require('./middlewares/auth');
```

Luego, creamos una ruta protegida que utilizará este middleware:

```
app.get('/protected-route', authenticateToken, (req, res) => {
  res.send("Esta es una ruta protegida");
});
```

Con esta configuración, cualquier solicitud a la ruta /protected-route pasará primero por el middleware authenticateToken, que verificará si el token proporcionado es válido antes de permitir el acceso.

¿Cómo probar la autenticación con Postman?

Para verificar que nuestro sistema de autenticación funciona correctamente, podemos utilizar Postman para realizar solicitudes a nuestra API:

1. Crear una nueva solicitud GET a la ruta protegida (<http://localhost:3000/protected-route>)
2. En la pestaña "Headers", agregar un encabezado llamado "Authorization" con el valor "Bearer [token]"
3. Enviar la solicitud

Si no proporcionamos un token o si el token es inválido, recibiremos un error de acceso denegado. En cambio, si el token es válido, podremos acceder a la ruta protegida.

Comportamiento esperado

- **Sin token:** Respuesta 401 - "Access denied. No token provided."
- **Token inválido:** Respuesta 403 - "Invalid token"
- **Token válido:** Acceso a la ruta protegida

Es importante destacar que para obtener un token válido, necesitaremos implementar rutas adicionales para el registro y la autenticación de usuarios, donde generaremos los tokens JWT después de verificar las credenciales.

La implementación de autenticación con JWT es un paso fundamental para proteger nuestras aplicaciones web. Este sistema nos permite verificar la identidad de los usuarios de manera eficiente y segura, controlando el acceso a recursos específicos. ¿Has implementado alguna vez este tipo de autenticación en tus proyectos? Comparte tu experiencia en los comentarios.